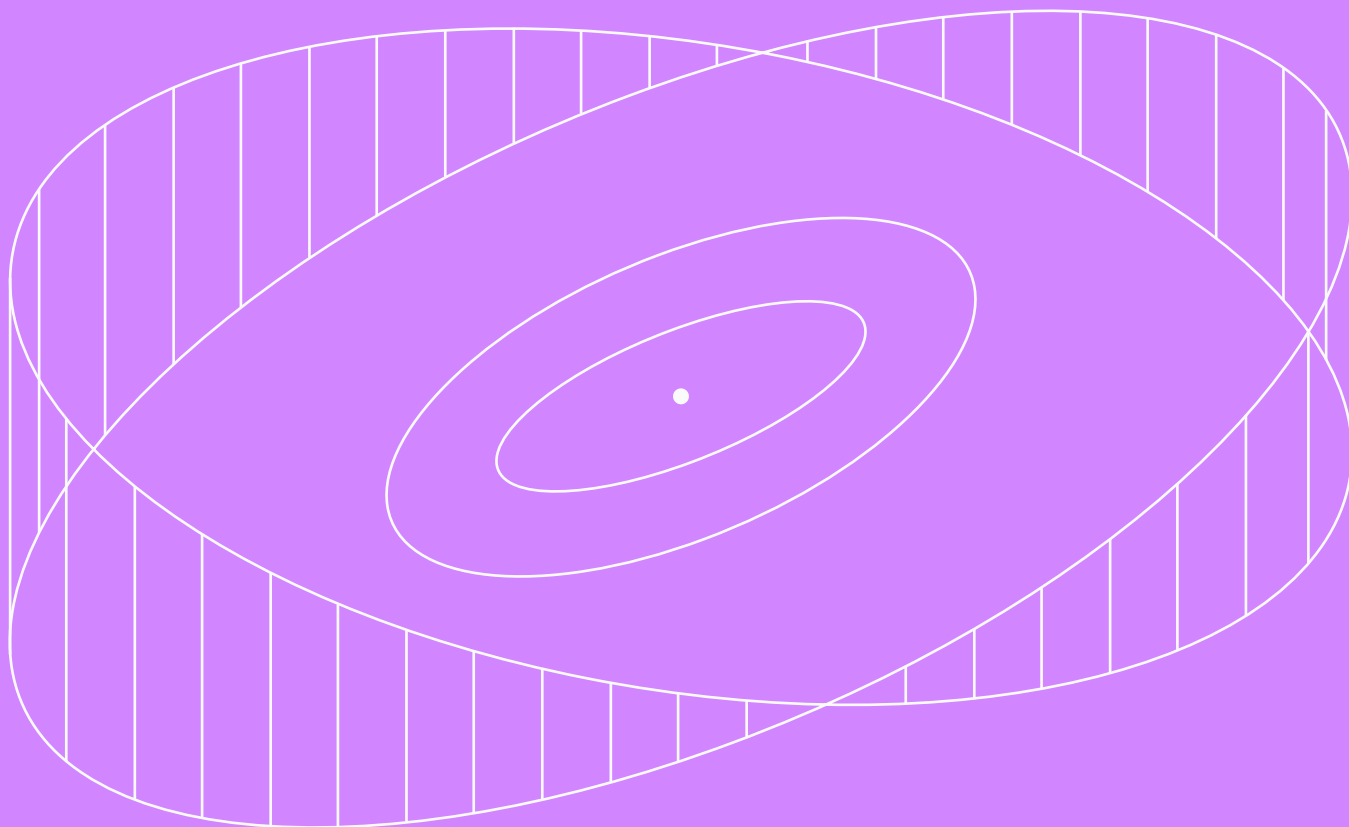


Блокировки

ВИДЫ БЛОКИРОВОК



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 12

В современных СУБД, таких как PostgreSQL, изоляция базируется на двух независимых механизмах. Первый — это MVCC, многоверсионность. Это элегантный, оптимистичный механизм, главная мантра которого: «Читатели не блокируют писателей, а писатели не блокируют читателей». Если возникает конфликт чтения и записи, база просто плодит новые версии строк. Никакого ожидания, никаких очередей. Но когда две транзакции пытаются изменить одни и те же данные или когда кто-то пытается изменить схему таблицы, оптимизм заканчивается. В игру вступает Lock Manager, диспетчер блокировок. Это пессимистичный, жёсткий механизм выстраивания очередей.



Если MVCC — это развязка на шоссе, то Lock Manager — это узкий мост с одним светофором. И именно этот светофор мы сейчас разберём на винтики.

Блокировка физически — это хеш-таблица в Shared Memory, разделяемой памяти сервера. В ней есть структура процесса — твоё соединение, структура ресурса, например, таблицы и связующее звено, которое говорит, что конкретный процесс держит конкретную таблицу в определённом режиме

Здесь кроется главный архитектурный инсайт:

ИНСАЙТ

Lock Manager работает только с высокоуровневыми объектами — таблицами, страницами, транзакциями. Он никогда не хранит блокировки конкретных строк.

Представь, что обновляешь миллиард строк. Если бы база создала миллиард записей в Lock Manager, она бы мгновенно исчерпала всю оперативную память, и ядро Linux убило бы процесс базы данных.

Как тогда база понимает, что строка заблокирована? Представь, что первая транзакция спустилась на самое дно и заблокировала одну строку в таблице пользователей. В это же время вторая транзакция решает удалить эту таблицу целиком. Сканировать миллиард строк перед удалением — безумие. Для этого существует иерархия намерений. Прежде чем изменить строку, транзакция обязана постучаться наверх и повесить табличную блокировку намерения. Это маркер, который говорит: «Я не блокирую саму таблицу для других обновлений, но я заявляю, что работаю внутри неё. Не смейте её удалять или менять ей схему». Блокировки всегда берутся сверху вниз, каскадом.

Мы говорим о тяжёлых блокировках, которые подчиняются правилам транзакций, умеют ждать часами и разруливать дедлоки. Но что защищает саму хеш-таблицу диспетчера блокировок, когда два процессорных ядра пытаются записать в неё данные одновременно? Для этого используются легковесные блокировки — LWLocks и Spinlocks. Они берутся на наносекунды и не знают, что такое транзакции. Если ты видишь в мониторинге, что запросы висят с ожиданием LWLock — проблема не в плохом SQL-коде. Проблема в том, что база упёрлась в дисковую подсистему или конфигурацию памяти, и железо физически не успевает перекладывать байты.

Любой инцидент бизнес-логики начинается с матрицы совместимости. Посмотри на классический **UPDATE**. Две такие операции берут эксклюзивную блокировку на строку, но на уровне таблицы они берут блокировку Row Exclusive. И эти табличные блокировки между собой совместимы. Диспетчер пропустит обе транзакции, и конфликтовать они будут только на физическом уровне диска. А теперь посмотрим на Access Exclusive Lock. Эту блокировку запрашивают операции изменения схемы, DDL. Она конфликтует абсолютно со всем, даже с обычным чтением. Любое изменение схемы таблицы мгновенно останавливает не только запись, но и чтение в этой таблице. База данных просто замирает.

Что происходит, если возникает конфликт несовместимых блокировок? Транзакция переводится в статус сна и встаёт в очередь ожидания. И здесь кроется самая опасная ловушка: очередь строго подчиняется правилу FIFO — кто первый пришёл, тот первый и получил ресурс. База гарантирует честность.

Представь ситуацию: висит долгий аналитический запрос на чтение. Ты запускаешь безобидный быстрый **ALTER TABLE**. Он конфликтует с чтением и встает в очередь. За ним приходят сотни коротких клиентских запросов от обычных пользователей. Хотя эти клиентские запросы совместимы с аналитикой, они не могут обогнать твой **ALTER TABLE** в очереди. СУБД запрещает обгон, чтобы не создать вечное голодание для изменения схемы. За несколько секунд в эту мертвую пробку упрутся все свободные коннекты твоего пулера. Микросервисы остаются без связи с БД, и приложение падает. Это классический каскадный сбой, который начинается с базового правила честной очереди.



Главный миф, который ломает архитектуру — это страх перед количеством заблокированных строк.

В некоторых базах данных, например в MS SQL Server, есть механизм эскалации блокировок. Если ты заблокируешь миллион строк, менеджер блокировок в оперативной памяти переполнится, и база принудительно заблокирует всю таблицу целиком. В PostgreSQL эскалации блокировок не существует. Ты можешь заблокировать одну строку или миллиард строк — потребление разделяемой памяти сервера не изменится ни на байт.

PostgreSQL пишет информацию о строковых блокировках не в оперативную память, а прямо в файлы данных. В заголовке каждой физической строки есть системное поле xmax. Туда записывается ID транзакции, которая эту строку заблокировала. А рядом, в поле infomask, ставится специальный битовый флаг, который говорит системе: «Эта транзакция не удалила строку, она просто повесила на неё замок». Это даёт нам бесконечную масштабируемость, но у этого есть цена — даже простой **SELECT FOR UPDATE** вызывает физическую операцию записи на диск.

Классический пример применения явной блокировки — паттерн Read-Modify-Write, который приводит к аномалии потерянного обновления. Если две транзакции одновременно прочитают баланс 100 и прибавят к нему единицу, в базе окажется 101 вместо 102. Классическая защита от этой гонки — конструкция **SELECT . . . FOR UPDATE**. Такое чтение берёт эксклюзивный замок на строку. Вторая транзакция, попытавшаяся прочитать эту же строку для обновления, наткнётся на чужой xmax и повиснет в очереди ожидания. Мы теряем параллелизм, мы заставляем процессы ждать, но мы гарантируем стопроцентную консистентность математики.

Однако использование голого **FOR UPDATE** в современном высоконагруженном приложении — это часто маркер низкой квалификации. Представь, что таблица заказов ссылается на таблицу пользователей через внешний ключ. Если ты сделаешь **FOR UPDATE** для пользователя, ты жёстко заблокируешь всю строку. Если параллельно другой микросервис попытается создать заказ для этого пользователя, он намертво зависнет. Ему нужно проверить внешний ключ, а для этого нужно прочитать ключ пользователя, который был заблокирован.

Элитные системные инженеры используют хирургически точный инструмент:

FOR NO KEY UPDATE. Эта команда говорит базе данных: «Я собираюсь изменить эту строку, но я клянусь, что не трону уникальные поля и первичный ключ». Это позволяет базе данных выдать тебе замок на изменение бизнес-данных, но при этом разрешить параллельным транзакциям безопасно создавать зависимые записи по внешнему ключу, вообще не дожидаясь окончания твоего запроса.

И здесь мы подходим к самому опасному побочному эффекту реляционных баз — скрытым блокировкам ссылочной целостности. Внешние ключи превращают изолированные таблицы в единый транзакционный граф. Когда ты делаешь обычный безобидный **INSERT** в таблицу заказов, PostgreSQL неявно, под капотом, берёт разделяемую блокировку **FOR KEY SHARE** на связанную строку в таблице пользователей. База обязана гарантировать, что пользователь не исчезнет, пока создаётся заказ. Тебе нужно всегда анализировать DDL-схему, потому что твои транзакции могут блокировать друг друга даже тогда, когда они физически пишут в абсолютно разные таблицы.

И наконец, как правильно работать с конкурентностью, если мы хотим построить очередь задач прямо в базе данных? Классический подход убьёт систему: если десять воркеров сделают **SELECT ... FOR UPDATE LIMIT 1**, первый воркер заберёт первую задачу, а остальные девять повиснут в ожидании этой же самой первой строки, создав глухое бутылочное горлышко.

Единственный архитектурно верный паттерн для очередей — это использование модификатора **SKIP LOCKED**. Он приказывает движку: «Если строка уже заблокирована кем-то другим, не жди в очереди. Просто проигнорируй её и иди к следующей свободной строке». Воркеры мгновенно разбирают задачи из таблицы без единой секунды простоя. Мы полностью избавляемся от очередей ожидания и получаем максимальную пропускную способность параллельной обработки, на которую способно твоё железо.

Тебе необходимо усвоить одно правило: изменение структуры таблицы, которое в тестовой среде выполняется за миллисекунду, может намертво положить кластер под нагрузкой. Любое изменение системного каталога, например, изменение типа колонки, требует абсолютной монополии. База данных должна гарантировать, что параллельный запрос не прочтает мусор или не вызовет ошибку сегментации памяти в ядре. Для этого накладывается Access Exclusive Lock. Как мы помним из матрицы совместимости, этот замок конфликтует абсолютно со всеми операциями, включая обычное чтение. Пока выполняется такой DDL-запрос, таблица физически недоступна.

Именно поэтому системные инженеры работают в парадигме Zero-Downtime миграций. Ты не имеешь права просто запустить **CREATE INDEX** на высоконагруженной боевой базе. Классическое создание индекса берёт Share Lock, который разрешает чтение, но намертво блокирует любые DML-операции — вставки, обновления и удаления — на всё время построения дерева индекса. Элитный инженер всегда платит временем процессора, чтобы сохранить доступность системы, и использует команду **CREATE INDEX CONCURRENTLY**. Она выполняет несколько проходов по таблице, работает дольше, но берёт слабую блокировку, которая не мешает ни чтению, ни записи. То же самое касается структуры: добавление новой колонки без значений или с константным дефолтом в современных версиях PostgreSQL выполняется безопасно, обновляя только метаданные, тогда как изменение существующего типа данных приведёт к переписыванию файлов на диске и многочасовой блокировке.

Но самый коварный убийца продакшена — это не долгая миграция. Это быстрая миграция, которая попала в ловушку очереди ожидания. Разберём сценарий каскадного сбоя. Аналитик запускает тяжёлый отчёт, который будет читать данные десять минут. В этот момент выкатывается релиз, и CI/CD-пайплайн запускает твой «быстрый» `ALTER TABLE`. Он запрашивает Access Exclusive Lock, видит конфликт с читающим отчётом аналитика и послушно встаёт в очередь ожидания (Wait Queue). И тут на сервер приходят тысячи обычных пользователей с миллисекундными запросами на чтение.

Из-за строгого правила FIFO, быстрые клиентские запросы не могут обогнать твой `ALTER TABLE` в очереди. Они полностью совместимы с отчётом аналитика, но они несовместимы с твоей миграцией, которая стоит перед ними. Возникает «мёртвая пробка». Тысячи простых `SELECT` зависают, ожидая пока пройдёт `ALTER TABLE`, который сам ожидает завершения десятиминутного отчёта. За пару секунд свободные соединения пулера заканчиваются, приложение начинает отдавать 504 Gateway Timeout, и система падает. «Быстрый» DDL убил кластер, даже не начав выполняться.

Единственный паттерн SRE для решения этой проблемы — это предохранитель `lock_timeout`. Никакие скрипты изменения схемы не имеют права запускаться на боевой базе без установки этого параметра в сессии миграции. Ты устанавливаешь тайм-аут блокировки, скажем, в две секунды. Если база нагружена и твоя миграция натывается на долгий запрос, создавая пробку, через две секунды сработает предохранитель. Транзакция миграции аварийно завершится с фатальной ошибкой, очередь мгновенно рассосётся, пропустив скопившийся пользовательский трафик. Запомни главное правило: упавшая миграция — это норма, ты перезапустишь её ночью. А вот упавший из-за заблокированной очереди продакшен — это непростительная ошибка проектирования.



Взаимоблокировка — это не баг базы данных. Это архитектурный провал разработчика. База данных работает идеально, она математически точно защищает твои ресурсы. А вот код приложения оказался не готов к конкурентной среде.

Локальный диспетчер блокировок видит только линейные очереди — кто за кем стоит. Но когда транзакция А захватывает первый ресурс и встаёт в очередь ко второму, который уже удерживает транзакция Б, а транзакция Б в это же время пытается захватить первый ресурс, возникает топологический цикл в графе ожиданий. Образуется смертельное объятие. Никто из них никогда не сделает `COMMIT`, потому что они будут ждать друг друга вечно.

Возникает резонный вопрос: почему база данных не обрывает дедлок моментально? Почему она ждёт? Ответ кроется в цене алгоритма обнаружения. Поиск циклов в направленном графе — это крайне ресурсоёмкая операция. Если твой кластер обслуживает десять тысяч запросов в секунду и при каждой блокировке СУБД будет строить граф и искать циклы, процессор просто сгорит.

Поэтому PostgreSQL использует алгоритм ленивой проверки. По умолчанию база считает, что любое ожидание — это не дедлок, а просто медленный соседний запрос. Твой процесс покорно засыпает. В этот момент включается таймер `deadlock_timeout`, который по умолчанию равен одной секунде. Если через секунду замок не освободился, СУБД понимает, что дело плохо.

Она «просыпается», строит граф ожиданий и ищет цикл. Если цикл найден, база выбирает жертву и принудительно убивает одну из транзакций, чтобы разорвать связь. Приложение получает фатальную ошибку "deadlock detected", а вторая транзакция разблокируется.

Вывод для инженера здесь неутешительный. Дедлоки не только обрывают твою бизнес-логику. Они заставляют драгоценные соединения простаивать в пуле как минимум одну лишнюю секунду. Если у тебя в системе регулярно случаются взаимоблокировки, твоё приложение будет не просто сыпать ошибками, оно начнёт деградировать от истощения свободных коннектов.

Пытаться ловить ошибку через блоки try/catch и повторять транзакцию в коде — это костыль, который маскирует плохую архитектуру. Тебе следует предотвращать дедлоки математически. Единственное правильное решение — это глобальный лексикографический порядок захвата ресурсов.

Представь микросервис, который получает массив ID для обновления: 5, 1 и 9. Если один поток начнёт обновлять их прямо в этом порядке, а параллельный поток получит из сети массив 9, 5, 1 и начнёт обновлять со своей стороны, ты гарантированно получишь дедлок на пересечении. Золотое правило транзакционной инженерии гласит: перед любым массовым обновлением или захватом строк через **SELECT FOR UPDATE** приложение обязано отсортировать ресурсы.

Если оба потока отсортируют массивы в памяти приложения и начнут захватывать блокировки строго по возрастанию: 1, 5, затем 9, то цикл станет невозможен. Вторая транзакция попытается взять первый ID, увидит блокировку от первой транзакции и просто подождёт. Никакого графа пересечений не возникнет. Смертельное обьятие превратится в обычную, абсолютно безопасную очередь ожидания. Ты устраняешь дедлоки не настройками базы данных, а одной строчкой кода с сортировкой в твоём микросервисе.

Мы живём в эпоху микросервисов, поэтому сейчас мы выходим на макро-уровень и поговорим о суровой эксплуатации — Day-2 Operations. Начнём с самого страшного инцидента в распределённых системах — двухфазного коммита, или протокола 2PC. Как списать деньги в базе биллинга и товар в базе склада абсолютно синхронно? Микросервис-координатор отправляет обеим базам команду **PREPARE TRANSACTION**. В этот момент PostgreSQL захватывает все нужные жёсткие блокировки, записывает данные в WAL и делает страшную вещь: он отвязывает эти блокировки от твоего TCP-соединения. База говорит: «Я готова, я держу ресурсы, жду твоей финальной команды».

А сейчас твой микросервис падает с нехваткой памяти, так и не отправляя финальную команду на коммит. Соединение разрывается. Обычно при обрыве связи база делает автоматический откат, но не в случае с **PREPARE**. В памяти PostgreSQL навсегда повисает транзакция-сирота. Её нет в списке активных процессов, но она держит эксклюзивные блокировки и, что самое страшное, останавливает горизонт видимости для всей базы. Старые версии строк перестают удаляться фоновой очисткой, диски переполняются, и кластер умирает от разбухания таблиц. Как системному инженеру тебе необходимо настроить жёсткий алерт на системное представление **pg_prepared_xacts**. Любая запись старше десяти минут там — это пожар, требующий ручного выполнения команды **ROLLBACK PREPARED**.

Иногда микросервисам нужна простая синхронизация, например, чтобы только один экземпляр фонового воркера запускал тяжёлый отчёт. Вместо того чтобы поднимать Redis, ты можешь использовать саму базу как распределённый мьютекс через Advisory Locks — совещательные блокировки. Ты блокируешь не таблицу, а просто 64-битное число. Но здесь кроется фатальная ловушка пулеров соединений, таких как PgBouncer. Если ты используешь блокировку на уровне сессии, и твой микросервис возвращает соединение в пул, блокировка остается висеть на этом коннекте. Следующий микросервис, получивший это соединение из пула, неожиданно для себя окажется владельцем чужого замка.



Золотое правило: в веб-приложениях с транзакционными пулерами используй только транзакционные совещательные блокировки — `pg_advisory_xact_lock`. Они гарантированно исчезают при коммите или откате.

Теперь представь: три часа ночи, дежурство, кластер не отвечает, график запросов пробил дно. Ты открываешь консоль. Матрицы совместимости и теория графов тебя сейчас не спасут. Тебе нужны глаза. Эти глаза — системные представления `pg_stat_activity` и `pg_locks`. Первое показывает, кто и что делает прямо сейчас. Если ты видишь сотни процессов со статусом ожидания Lock, значит, образовалась мёртвая пробка. Второе представление — это прямой дамп памяти менеджера блокировок.



Главная ошибка неопытного инженера в этот момент — начать убивать все зависшие запросы. Это бессмысленно, ты просто убиваешь невинных жертв, которые стоят в честной очереди ожидания.

Тебе нужно найти корень проблемы — того единственного, кто держит ресурс. В PostgreSQL для этого есть встроенная функция `pg_blocking_pids`. Ты смотришь на зависший процесс и спрашиваешь базу: «Кто его блокирует?». Если ты видишь, что двести процессов ждут один единственный PID, ты не трогаешь эти двести процессов. Ты выполняешь `pg_terminate_backend` для корневого блокиратора. Очень часто им оказывается процесс со статусом «idle in transaction» — кто-то открыл транзакцию, обновил строку, а затем приложение пошло делать долгий HTTP-запрос в стороннюю систему, оставив базу висеть на замках.

Я хочу дать тебе подсказки, которые ты можешь применять при проектировании любой системы.

- Первое: держи транзакции короткими. Никаких сетевых вызовов, общения с внешними API или gRPC внутри открытого блока `BEGIN-COMMIT`. Транзакция — это только работа с диском и памятью.
- Второе: перед массовыми обновлениями всегда сортируй идентификаторы ресурсов лексикографически, чтобы математически исключить дедлоки.
- Третье: ни одно изменение структуры базы не едет на продакшен без предохранителя `lock_timeout`. Четвёртое: для создания высоконагруженных очередей обработки используй только паттерн `SKIP LOCKED`.